

Lab Assignment-4: Trees & Graphs

Basics of Data Structures (ETCCBD201)
School of Engineering & Technology

Task 1: Binary Search Tree (BST)

1.1 Why Inorder Traversal Prints a BST in Sorted Order

A BST enforces the invariant: for every node N , all keys in the left subtree are less than $N.key$ and all keys in the right subtree are greater than $N.key$. Inorder traversal visits nodes in the sequence Left $\rightarrow$ Root $\rightarrow$ Right. Because every node to the left is smaller and every node to the right is larger, this recursive ordering guarantees that elements are visited in ascending order from the globally smallest to the globally largest key.

1.2 Time Complexity Discussion

Operation	Average Case	Worst Case (Skewed)	Reason
Insert	$O(\log n)$	$O(n)$	Tree height $h = \log n$ (balanced) or n (skewed)
Search	$O(\log n)$	$O(n)$	Same as above; compare one node per level
Delete	$O(\log n)$	$O(n)$	Must locate node first, then fix subtree
Inorder	$O(n)$	$O(n)$	Must visit every node exactly once

Average case $O(\log n)$: In a balanced BST the height is approximately $\log_2(n)$, so each operation compares only $\log n$ nodes before reaching a leaf. **Worst case $O(n)$:** If elements are inserted in sorted order (e.g. 1, 2, 3, ... n) the tree degenerates into a linked list of height n , making every operation $O(n)$. Self-balancing BSTs (AVL, Red-Black) guarantee $O(\log n)$ always.

1.3 Test Plan Results

Operation	Input	Output / Inorder after operation
Insert	[50,30,70,20,40,60,80]	[20,30,40,50,60,70,80]
Search	20	True
Search	90	False
Delete (leaf – no child)	20	[30,40,50,60,70,80]
Insert then delete (one child)	65, then delete 60	[30,40,50,65,70,80]
Delete (two children)	30	[40,50,65,70,80]
Delete root (two children)	50	[40,65,70,80]

Task 2: Graph Representation + BFS and DFS

2.1 Graph Used (Directed, Weighted – 6 Nodes, 9 Edges)

$A \rightarrow B(2)$, $A \rightarrow C(4)$, $B \rightarrow D(7)$, $B \rightarrow E(3)$, $C \rightarrow E(1)$, $C \rightarrow F(8)$, $D \rightarrow F(5)$, $E \rightarrow D(2)$, $E \rightarrow F(6)$

2.2 Adjacency List Output

A → [B(w=2), C(w=4)]

B → [D(w=7), E(w=3)]

C → [E(w=1), F(w=8)]

D → [F(w=5)]

E → [D(w=2), F(w=6)]

F → []

2.3 BFS vs DFS – Explanation

Aspect	BFS (Breadth-First Search)	DFS (Depth-First Search)
Strategy	Explore all neighbours at current level before going deeper	Go as deep as possible along one path before backtracking
Data struct	Queue (FIFO)	Stack (LIFO) or call stack (recursion)
Traversal from A	A → B → C → D → E → F	A → B → D → F → E → C
Real-world use case	Shortest path in unweighted graphs (e.g. social-network recommendation)	Only one shortest path (e.g. topological sort, solving mazes, DFS spanning tree)

2.4 Observations

BFS guarantees the shortest number of hops from the source in an unweighted graph because it explores level by level. DFS may reach distant nodes quickly but does not guarantee shortest paths. For this directed weighted graph the adjacency list representation is memory-efficient: it only stores existing edges, whereas an adjacency matrix would allocate $n^2 = 36$ cells for 6 nodes regardless of edge count.

References

- Cormen, T. H. et al. *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- Python documentation – collections.deque: <https://docs.python.org/3/library/collections.html>
- GeeksforGeeks – Binary Search Tree (conceptual reference): <https://www.geeksforgeeks.org/binary-search-tree-data-structure/>